

TeamFlow — Feature Branches & Implementation Roadmap

GitHub branch strategy • Folder structure • Skill gaps • Detailed task lists

1. Dividing the Project into Features

TeamFlow is split into focused, independently deliverable features. Each feature maps to one short-lived GitHub branch. This demonstrates real-world Git workflow (feature branches → PR → merge to main) and keeps every commit interview-defensible.

- **Auth & User Management** — Register, login, logout, JWT, password hashing
- **Workspaces & Multi-tenancy** — Create workspaces, membership, OWNER/ADMIN/MEMBER roles
- **Projects CRUD** — Create / list / update / delete projects inside a workspace
- **Tasks Core** — Task model, status, priority, assignee, due date, basic CRUD
- **Task Business Rules** — Status transitions, activity log, optimistic concurrency (version)
- **Members Management** — Invite, change role, remove member, last-owner protection
- **Labels & Comments** — Workspace labels, task-label M2M, comments on tasks
- **Search / Filter / Pagination** — Query params, Prisma filters, indexes, sorting
- **Frontend Core UI** — Auth pages, workspace list, project board, task detail
- **Quality & Hardening** — Validation, global errors, logging, unit/integration/E2E tests, CI

2. GitHub Branches to Create (with short descriptions)

Start from **main**. Create these feature branches from main. Keep them short-lived. Open a PR for every merge.

main

Production-ready code. Only merged, reviewed, tested code lives here.

feature/auth

User registration, login, logout, JWT issuance, password hashing, AuthGuard.

feature/workspaces

Workspace model, create/list workspaces, WorkspaceMember join table + roles enum.

feature/projects

Project model + full CRUD endpoints with membership & role checks.

feature/tasks-core

Task model (status, priority, assignee, due_date, version) + create/list endpoints.

feature/tasks-rules

PATCH task, status transitions, activity log inside transaction, optimistic concurrency.

feature/members

Invite / change-role / remove member. OWNER/ADMIN only. Last-owner protection.

feature/labels-comments

Labels at workspace level, task_labels M2M, comments on tasks.

feature/search-filter

GET /tasks with status/priority/assignee/q/page/limit/sort + indexes.

feature/frontend-core

Next.js auth pages, workspace list, project list/board, task detail UI.

feature/quality

Global exception filter, structured logging, Jest unit/integration, Playwright E2E, GitHub Actions.

3. Complete Folder & File Structure

Monorepo recommended (one GitHub repo). Adjust if you prefer two separate repos.

```
teamflow/
├── README.md
├── .gitignore
├── .env.example
├── docker-compose.yml
├── package.json                # optional root workspace scripts
├──
├── backend/                   # NestJS + TypeScript
│   ├── package.json
│   ├── tsconfig.json
│   ├── nest-cli.json
│   ├── Dockerfile
│   ├── .env.example
│   ├── prisma/
│   │   ├── schema.prisma
│   │   └── migrations/
│   ├── src/
│   │   ├── main.ts
│   │   ├── app.module.ts
│   │   ├── common/
│   │   │   ├── decorators/      # @Public(), @Roles(), @CurrentUser()
│   │   │   ├── filters/        # Global exception filter
│   │   │   ├── guards/         # JwtAuthGuard, RolesGuard
│   │   │   ├── interceptors/
│   │   │   ├── pipes/
│   │   │   ├── dto/            # shared pagination / error shapes
│   │   │   └── auth/
│   │       ├── auth.module.ts
│   │       ├── auth.controller.ts
│   │       ├── auth.service.ts
│   │       └── dto/
│   │           ├── register.dto.ts
│   │           ├── login.dto.ts
│   │           └── strategies/   # jwt.strategy.ts
│   │       └── users/
│   │           ├── users.module.ts
│   │           ├── users.service.ts
│   │           └── users.controller.ts (optional)
│   │       └── workspaces/
│   │           ├── workspaces.module.ts
│   │           ├── workspaces.controller.ts
│   │           ├── workspaces.service.ts
│   │           └── dto/
│   │       └── projects/
│   │           ├── projects.module.ts
│   │           ├── projects.controller.ts
│   │           ├── projects.service.ts
│   │           └── dto/
│   │       └── tasks/
│   │           ├── tasks.module.ts
│   │           ├── tasks.controller.ts
│   │           ├── tasks.service.ts
│   │           └── dto/
│   │       └── members/
│   │           ├── members.module.ts
│   │           ├── members.controller.ts
│   │           ├── members.service.ts
│   │           └── dto/
│   │       └── labels/
│   │           ├── labels.module.ts
│   │           ├── labels.controller.ts
│   │           ├── labels.service.ts
│   │           └── dto/
│   │       └── comments/
│   │           ├── comments.module.ts
│   │           └── comments.controller.ts
```

```

■ ■ ■ ■ ■ comments.service.ts
■ ■ ■ ■ ■ dto/
■ ■ ■ ■ ■ activity/
■ ■ ■ ■ ■ activity.module.ts
■ ■ ■ ■ ■ activity.service.ts
■ ■ ■ ■ ■ activity.controller.ts
■ ■ ■ ■ ■ prisma/
■ ■ ■ ■ ■ prisma.module.ts
■ ■ ■ ■ ■ prisma.service.ts
■ ■ ■ ■ ■ test/ # unit + integration
■ ■ ■ ■ ■ auth.service.spec.ts
■ ■ ■ ■ ■ tasks.service.spec.ts
■ ■ ■ ■ ■ ...
■
■ ■ ■ ■ ■ frontend/ # Next.js App Router + TS + Tailwind
■ ■ ■ ■ ■ package.json
■ ■ ■ ■ ■ tsconfig.json
■ ■ ■ ■ ■ next.config.js
■ ■ ■ ■ ■ tailwind.config.ts
■ ■ ■ ■ ■ postcss.config.js
■ ■ ■ ■ ■ Dockerfile (optional)
■ ■ ■ ■ ■ .env.example
■ ■ ■ ■ ■ public/
■ ■ ■ ■ ■ src/
■ ■ ■ ■ ■ app/
■ ■ ■ ■ ■ layout.tsx
■ ■ ■ ■ ■ page.tsx
■ ■ ■ ■ ■ login/page.tsx
■ ■ ■ ■ ■ register/page.tsx
■ ■ ■ ■ ■ workspaces/
■ ■ ■ ■ ■ page.tsx
■ ■ ■ ■ ■ [id]/page.tsx
■ ■ ■ ■ ■ projects/
■ ■ ■ ■ ■ [id]/page.tsx
■ ■ ■ ■ ■ tasks/
■ ■ ■ ■ ■ [id]/page.tsx
■ ■ ■ ■ ■ components/
■ ■ ■ ■ ■ ui/
■ ■ ■ ■ ■ TaskCard.tsx
■ ■ ■ ■ ■ ProjectBoard.tsx
■ ■ ■ ■ ■ ...
■ ■ ■ ■ ■ lib/
■ ■ ■ ■ ■ api-client.ts
■ ■ ■ ■ ■ auth.ts
■ ■ ■ ■ ■ types/
■ ■ ■ ■ ■ index.ts
■ ■ ■ ■ ■ e2e/
■ ■ ■ ■ ■ happy-path.spec.ts
■
■ ■ ■ ■ ■ .github/
■ ■ ■ ■ ■ workflows/
■ ■ ■ ■ ■ ci.yml

```

4. Skill Gaps & What to Learn Before Starting

Highest-priority gaps from the TeamFlow guide. Get comfortable with these before or during week 1.

Highest Priority (learn first)

- TypeScript strict mode — interfaces, types, generics, DTOs, type narrowing, strictNullChecks
- Deeper JavaScript — closures, event loop, this binding, prototypes, async/await patterns
- Node.js + NestJS — modules, controllers, services, DI, guards, pipes, filters
- PostgreSQL + relational design — normalization, FKs, unique constraints, indexes, migrations
- Prisma ORM — schema, relations, migrations vs db push, transactions
- OOP principles applied in TypeScript (SOLID in practice)
- AuthN/AuthZ — bcrypt/argon2, JWT, guards, RBAC, never trust the frontend
- Software architecture & clean layering (controllers vs services vs data access)
- Testing mindset — unit (Jest), integration, E2E (Playwright)
- Git workflow — feature branches, meaningful commits, PRs, GitHub Actions basics
- Basic system-design talking points (multi-tenancy, optimistic concurrency, ACID)

Secondary (touch through the project)

- Docker Compose for real multi-service apps
- Simple cloud deploy (Vercel + Render/Railway)
- API docs (Swagger/OpenAPI optional)
- Input validation (class-validator)
- Structured logging & error handling shapes
- Pagination, filtering, sorting
- Optimistic concurrency (version column + 409 Conflict)
- Observability basics

Recommended pre-reading order: TypeScript handbook (strict) → NestJS official docs (modules + guards) → Prisma docs (schema + migrations) → JWT + bcrypt concepts → Jest + Playwright quickstarts.

5. Feature Branches — Detailed Task Lists & Files to Create

Use each section as a checklist. Create the listed files on that branch. After finishing: self-review → open PR → merge to main → delete branch.

Branch: feature/auth

Goal: Working register/login/logout + JWT + password hashing + protected route.

Tasks

- Scaffold NestJS backend with TypeScript strict mode
- Install & configure @nestjs/jwt, passport, bcrypt, class-validator, class-transformer
- Create User model in Prisma (id, email, password_hash, name, timestamps)
- Write migration and apply it
- Implement AuthService: register (hash password), login (compare + sign JWT)
- Implement AuthController: POST /auth/register, POST /auth/login, POST /auth/logout
- Create DTOs with validation (RegisterDto, LoginDto)
- Create JwtStrategy + JwtAuthGuard
- Add @Public() decorator and protect a test route
- Write unit tests for password hashing and AuthService
- Update .env.example with JWT_SECRET, DATABASE_URL

Key files to create

- backend/prisma/schema.prisma (User model)
- backend/src/auth/auth.module.ts, auth.controller.ts, auth.service.ts
- backend/src/auth/dto/register.dto.ts, login.dto.ts
- backend/src/auth/strategies/jwt.strategy.ts
- backend/src/common/guards/jwt-auth.guard.ts
- backend/src/common/decorators/public.decorator.ts
- backend/src/users/users.service.ts
- backend/test/auth.service.spec.ts

Branch: feature/workspaces

Goal: Multi-tenant foundation — workspaces + membership + roles.

Tasks

- Add Workspace + WorkspaceMember models to Prisma (role enum: OWNER | ADMIN | MEMBER)
- Unique constraint on (workspace_id, user_id)
- Create migration
- WorkspacesService: create (auto-add creator as OWNER), list for current user, get by id
- Membership checks: helper that verifies user belongs to workspace
- WorkspacesController: GET/POST /workspaces, GET /workspaces/:id
- Protect all routes with JwtAuthGuard
- Return only workspaces the user is a member of

Key files to create

- backend/prisma/schema.prisma (Workspace, WorkspaceMember, Role enum)
- backend/src/workspaces/workspaces.module.ts
- backend/src/workspaces/workspaces.controller.ts
- backend/src/workspaces/workspaces.service.ts
- backend/src/workspaces/dto/create-workspace.dto.ts
- backend/src/common/decorators/current-user.decorator.ts

Branch: feature/projects

Goal: Project CRUD with workspace membership + role authorization.

Tasks

- Add Project model (workspace_id, name, description, status, timestamps, optional deleted_at)
- Migration
- ProjectsService: create, list by workspace, get, update, soft-delete
- Every method first checks membership and required role
- ProjectsController under /workspaces/:id/projects and /projects/:id

- DTOs with validation
- Return 403 when user is not a member

Key files to create

- backend/src/projects/projects.module.ts
- backend/src/projects/projects.controller.ts
- backend/src/projects/projects.service.ts
- backend/src/projects/dto/create-project.dto.ts, update-project.dto.ts

Branch: feature/tasks-core

Goal: Task model + basic create & list with assignee validation.

Tasks

- Add Task model (project_id, title, description, status, priority, assignee_id, due_date, version)
- Enums for Status (TODO, IN_PROGRESS, DONE, ...) and Priority
- Migration + indexes on project_id, status, assignee_id
- TaskService: create (validate assignee is workspace member), list by project
- TaskController: POST/GET /projects/:id/tasks, GET /tasks/:id
- DTOs

Key files to create

- backend/src/tasks/tasks.module.ts
- backend/src/tasks/tasks.controller.ts
- backend/src/tasks/tasks.service.ts
- backend/src/tasks/dto/create-task.dto.ts

Branch: feature/tasks-rules

Goal: Updates, business rules, activity log, optimistic concurrency.

Tasks

- Implement PATCH /tasks/:id with partial updates
- Enforce allowed status transitions (if you define a state machine)
- Add version column check: if client version $\neq$ current $\rightarrow$ 409 Conflict
- Create ActivityLog model + ActivityService
- Write activity entry inside the same Prisma transaction as the task change
- Centralize authorization helpers (membership + role) so services stay DRY

Key files to create

- backend/src/tasks/dto/update-task.dto.ts
- backend/src/activity/activity.module.ts, activity.service.ts
- backend/src/common/helpers/authorization.helper.ts

Branch: feature/members

Goal: Invite, change role, remove member with last-owner protection.

Tasks

- MembersService: invite (by email or userId), changeRole, remove
- Only OWNER or ADMIN can manage members
- Prevent removing the last OWNER
- Controller: POST/PATCH/DELETE under /workspaces/:id/members
- DTOs + clear error messages (403 / 400)

Key files to create

- backend/src/members/members.module.ts
- backend/src/members/members.controller.ts
- backend/src/members/members.service.ts
- backend/src/members/dto/invite-member.dto.ts, change-role.dto.ts

Branch: feature/labels-comments

Goal: Labels at workspace level + comments on tasks.

Tasks

- Label model (workspace_id, name, color) + task_labels M2M
- Comment model (task_id, author_id, body, timestamps)
- LabelsService + CommentsService with membership checks
- Endpoints: CRUD labels under workspace, attach/detach to task, POST/GET comments on task

Key files to create

- backend/src/labels/labels.module.ts, labels.controller.ts, labels.service.ts, dto/
- backend/src/comments/comments.module.ts, comments.controller.ts, comments.service.ts, dto/

Branch: feature/search-filter

Goal: Powerful GET /tasks with query params + indexes.

Tasks

- Accept query params: status, priority, assignee, q (search), page, limit, sort
- Build Prisma where + orderBy + skip/take dynamically
- Add DB indexes on filtered columns
- Return { data, total, page, limit }
- Validate enum values; reject unknown params gracefully

Key files to create

- backend/src/tasks/dto/query-tasks.dto.ts
- Update tasks.service.ts (list method)
- Update prisma/schema.prisma (@@index)

Branch: feature/frontend-core

Goal: Usable UI for the core flows (not pixel-perfect).

Tasks

- Scaffold Next.js (App Router) + TypeScript + Tailwind
- API client (fetch wrapper) that attaches JWT
- Login & Register pages with form validation
- Store token safely (httpOnly cookie preferred)
- Workspace list page + create workspace form
- Project list / simple board inside a workspace
- Task detail page or drawer (status, priority, assignee, comments)
- Loading and error states on every data-fetch
- Protected-route pattern — backend still source of truth

Key files to create

- frontend/src/app/login/page.tsx, register/page.tsx
- frontend/src/app/workspaces/page.tsx, [id]/page.tsx
- frontend/src/app/projects/[id]/page.tsx
- frontend/src/app/tasks/[id]/page.tsx
- frontend/src/lib/api-client.ts, auth.ts
- frontend/src/components/TaskCard.tsx, ProjectBoard.tsx, ui/*
- frontend/src/types/index.ts

Branch: feature/quality

Goal: Production-quality hardening + tests + CI.

Tasks

- Global exception filter → consistent { statusCode, message, error, details? }
- Structured logging (no secrets in logs)
- Unit tests for critical services (Auth, Tasks, Members) with mocked Prisma
- Integration tests for key endpoints (use test DB or transactions)
- One solid Playwright E2E: register → workspace → project → task → assign → comment
- GitHub Actions workflow: lint, test, build on every PR
- Final README with setup, architecture notes, how to run tests
- Code review pass: extract shared auth helpers, remove dead code, document decisions

Key files to create

- backend/src/common/filters/http-exception.filter.ts
- backend/test/*.spec.ts (unit + integration)
- frontend/e2e/happy-path.spec.ts
- .github/workflows/ci.yml
- README.md (full)

Interview tip: Be ready to explain why you chose feature branches, how RBAC is enforced on the backend, what optimistic concurrency solves, and how you structured the NestJS layers. Draw the ER diagram from memory.